

Plano de Implementação — Estruturas de Dados, Ordenação e Grafos

Checklist dividido em pequenos passos, um por estrutura/algoritmo. Cada estrutura de dados tem inserção, busca e remoção; cada algoritmo de ordenação tem apenas ordenação e busca (busca sequencial/binária sobre o array ordenado, quando fizer sentido); os algoritmos de grafos (Kruskal, Prim, Dijkstra) têm representação do grafo + laço principal + testes.

Sugestão de estrutura de pastas (seguindo o padrão já usado em tress/):

```
tress/  
  bst/           (já iniciado)  
  avl/  
  red-black/    (já iniciado)  
heaps/  
  priority-queue/  
sorting/  
  merge-sort/  
  quick-sort/  
  counting-sort/  
  insertion-sort/  
  selection-sort/  
graphs/  
  mst-kruskal/  
  mst-prim/  
  dijkstra/
```

1. Árvore Binária de Busca (BST)

- ☐ Definir struct do nó (struct Node { int key; struct Node *left, *right; })
- ☐ Implementar criação de nó / árvore vazia
- ☐ Implementar inserção (recursiva ou iterativa)
- ☐ Testar inserção com casos: árvore vazia, inserir à esquerda, à direita, chave duplicada
- ☐ Implementar busca (recursiva ou iterativa)
- ☐ Testar busca: chave existente, chave inexistente, árvore vazia
- ☐ Implementar remoção — caso folha
- ☐ Implementar remoção — caso um filho
- ☐ Implementar remoção — caso dois filhos (sucessor/predecessor in-order)
- ☐ Testar remoção com os três casos acima
- ☐ Implementar percurso in-order para depuração/validação da árvore
- ☐ Escrever main.c com menu ou bateria de testes automatizados

2. Árvore AVL

- ☐ Definir struct do nó com campo de altura (ou calcular altura sob demanda)
- ☐ Implementar cálculo de altura e fator de balanceamento
- ☐ Implementar rotação simples à direita
- ☐ Implementar rotação simples à esquerda
- ☐ Implementar rotação dupla esquerda-direita
- ☐ Implementar rotação dupla direita-esquerda
- ☐ Implementar inserção com rebalanceamento pós-inserção
- ☐ Testar inserção forçando cada um dos 4 casos de desbalanceamento
- ☐ Implementar busca (idêntica à BST)
- ☐ Testar busca: chave existente, inexistente, árvore vazia
- ☐ Implementar remoção com rebalanceamento pós-remoção
- ☐ Testar remoção forçando rebalanceamento em cascata (múltiplas rotações)
- ☐ Escrever main.c com testes que imprimem alturas/fatores de balanceamento

3. Árvore Rubro-Negra

- ☐ Definir struct do nó com campo de cor (usar sentinela NIL preto)
- ☐ Implementar rotação à esquerda e à direita
- ☐ Implementar inserção padrão de BST + pintura inicial do nó como vermelho
- ☐ Implementar fixup de inserção (casos 1, 2 e 3 do tio vermelho/preto)
- ☐ Testar inserção verificando as 5 propriedades rubro-negras após cada inserção
- ☐ Implementar busca (idêntica à BST, ignorando cor)
- ☐ Testar busca: chave existente, inexistente, árvore vazia
- ☐ Implementar transplante de subárvore (transplant)
- ☐ Implementar remoção padrão de BST adaptada (localizar sucessor, y-original-color)
- ☐ Implementar fixup de remoção (todos os casos: irmão vermelho, irmão preto com filhos pretos/vermelhos)
- ☐ Testar remoção verificando as 5 propriedades após cada remoção
- ☐ Escrever main.c com função de validação automática das propriedades RB

4. Heap / Fila de Prioridade

- ☐ Decidir representação (array binário 0-indexado ou 1-indexado)
- ☐ Implementar heapify-up (sift-up) e heapify-down (sift-down)
- ☐ Implementar inserção (push) com heapify-up
- ☐ Testar inserção: heap vazio, manter propriedade de heap após várias inserções
- ☐ Implementar busca — localizar elemento no array (busca linear, já que heap não é ordenado para busca eficiente)
- ☐ Testar busca: elemento existente, inexistente
- ☐ Implementar remoção do topo (extract-min/max) com heapify-down
- ☐ Implementar remoção de elemento arbitrário (trocar com último, remover, heapify em ambas direções)

- ☐ Testar remoção: heap com 1 elemento, remoção do topo, remoção de elemento do meio
- ☐ Escrever main.c implementando fila de prioridade completa (push/pop/peek) com testes

5. Algoritmos de Ordenação (ordenação + busca apenas)

Para cada algoritmo: implementar a ordenação, testar em casos-limite, depois implementar busca binária (ou sequencial, se indicado) sobre o array resultante.

5.1 Insertion Sort

- ☐ Implementar insertion sort in-place
- ☐ Testar: array vazio, 1 elemento, já ordenado, ordem reversa, com duplicatas
- ☐ Implementar busca binária sobre o array ordenado
- ☐ Testar busca: elemento presente, ausente, nas extremidades

5.2 Selection Sort

- ☐ Implementar selection sort in-place
- ☐ Testar: array vazio, 1 elemento, já ordenado, ordem reversa, com duplicatas
- ☐ Implementar busca binária sobre o array ordenado
- ☐ Testar busca: elemento presente, ausente, nas extremidades

5.3 Merge Sort

- ☐ Implementar função de merge de dois subarrays ordenados
- ☐ Implementar merge sort recursivo (divisão + merge)
- ☐ Testar: array vazio, 1 elemento, já ordenado, ordem reversa, com duplicatas
- ☐ Implementar busca binária sobre o array ordenado
- ☐ Testar busca: elemento presente, ausente, nas extremidades

5.4 Quick Sort

- ☐ Implementar função de particionamento (escolher estratégia de pivô)
- ☐ Implementar quick sort recursivo
- ☐ Testar: array vazio, 1 elemento, já ordenado, ordem reversa, com duplicatas, muitos elementos iguais
- ☐ Implementar busca binária sobre o array ordenado
- ☐ Testar busca: elemento presente, ausente, nas extremidades

5.5 Counting Sort

- ☐ Implementar contagem de frequências e array de posições acumuladas
 - ☐ Implementar construção do array ordenado a partir das contagens
 - ☐ Testar: array vazio, 1 elemento, valores repetidos, faixa de valores conhecida
 - ☐ Implementar busca binária (ou sequencial) sobre o array ordenado
 - ☐ Testar busca: elemento presente, ausente, nas extremidades
-

6. Grafos — Árvore Geradora Mínima e Caminho Mínimo

Estes são algoritmos (não estruturas com inserção/busca/remoção), então o checklist cobre representação do grafo, implementação do algoritmo e casos de teste.

6.1 Representação do grafo (pré-requisito comum)

- ☐ Definir estrutura de grafo ponderado não-direcionado (lista de arestas e/ou lista de adjacência)
- ☐ Implementar leitura/construção do grafo (a partir de array de arestas (u, v, peso))
- ☐ Implementar impressão do grafo para depuração

6.2 Kruskal (MST)

- ☐ Implementar estrutura Union-Find (Disjoint Set) com make-set, find
- ☐ Implementar union por rank/tamanho com compressão de caminho no find
- ☐ Implementar ordenação das arestas por peso (reaproveitar sort já implementado)
- ☐ Implementar laço principal de Kruskal (adicionar aresta se não forma ciclo)
- ☐ Testar: grafo conexo simples, grafo com arestas de peso igual, grafo desconexo (floresta geradora)
- ☐ Validar custo total da MST contra resultado calculado manualmente

6.3 Prim (MST)

- ☐ Implementar representação por lista de adjacência (necessária para Prim eficiente)
- ☐ Implementar fila de prioridade min (reaproveitar heap já implementado) com chave = menor peso de aresta até a árvore
- ☐ Implementar laço principal de Prim (extrair mínimo, relaxar arestas adjacentes)
- ☐ Testar: grafo conexo simples, grafo com pesos iguais, grafo denso vs. esparso
- ☐ Validar custo total da MST e comparar com resultado do Kruskal (devem coincidir)

6.4 Dijkstra (caminho mínimo)

- ☐ Implementar representação por lista de adjacência com pesos não-negativos
- ☐ Implementar array/estrutura de distâncias iniciais (infinito, exceto origem = 0)
- ☐ Implementar fila de prioridade min (reaproveitar heap já implementado) chaveada por distância
- ☐ Implementar laço principal (extrair mínimo, relaxar arestas adjacentes)
- ☐ Implementar reconstrução do caminho (array de predecessores + função de impressão do caminho)
- ☐ Testar: grafo simples com caminho único, grafo com múltiplos caminhos de custo igual, vértice inalcançável (distância infinita)
- ☐ Testar caso de origem = destino (distância zero, caminho trivial)

Ordem sugerida de execução

- BST → 2. AVL → 3. Rubro-Negra → 4. Heap/Fila de Prioridade →
- Insertion → Selection → Merge → Quick → Counting Sort →

- Representação de grafo → Kruskal → Prim → Dijkstra

Cada estrutura/algoritmo deve ser concluído (todos os checkboxes marcados) antes de avançar para o próximo, para reforçar o aprendizado incremental. Prim e Dijkstra reaproveitam a fila de prioridade (heap) implementada no passo 4 — vale revisá-la antes de começar o passo 6.